UNIVERSIDAD DE CARABOBO

FACULTAD EXPERIMENTAL DE CIENCIAS Y TECNOLOGIA

NAGUANAGUA, CARABOBO

PRACTICA #3

ESTUDIANTES:

FABIAN PEÑA C.I: 31.994.692
SANTIAGO ESCALONA C.I: 31.768.925

PROFESOR:

JOSE CANACHE

1) Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema que utiliza memory-mapped I/O (MMIO), los dispositivos de entrada y salida (E/S) no están conectados a través de puertos específicos, como en otros tipos de sistemas, sino que están organizados como una región especial de la memoria del sistema. En lugar de interactuar con dispositivos a través de instrucciones dedicadas como IN o OUT, el procesador accede a los dispositivos mediante las mismas instrucciones que utilizaría para leer y escribir en la memoria, como lw (load word) y sw (store word).

Organización de la memoria:

En un sistema MIPS32 con MMIO, los dispositivos de E/S se mapean dentro del espacio de direcciones de memoria, pero en rangos específicos que no se solapan con la memoria general utilizada por los programas. Dependiendo de la arquitectura, estos rangos suelen estar al final del espacio de direcciones de memoria, o en direcciones específicas definidas en el diseño del sistema. Por ejemplo:

Los registros de control de los dispositivos (como LuzControl o PresionControl) suelen ser mapeados en direcciones de memoria específicas.

Estos dispositivos están asignados a direcciones fijas, de modo que el procesador puede acceder a ellos como si fueran ubicaciones de memoria.

Región de memoria para mapear dispositivos:

Los dispositivos se suelen mapear en una región de memoria reservada. En sistemas MIPS32, esta región puede estar en un rango de direcciones como:

Direcciones altas en el espacio de direcciones de memoria (por ejemplo, entre 0xFFFF0000 y 0xFFFFFFFF).

También puede depender de la implementación específica del sistema o del sistema operativo que esté utilizando.

Implicaciones para las instrucciones lw y sw:

Las instrucciones lw y sw tienen implicaciones directas en el uso de MMIO:

`lw` (load word): Esta instrucción carga un valor de 32 bits desde una dirección de memoria especificada. En el contexto de MMIO, si la dirección corresponde a un dispositivo mapeado, lw leerá el valor actual del dispositivo (por ejemplo, el estado de un sensor o el dato que ha medido).

`sw` (store word): Esta instrucción escribe un valor de 32 bits en una dirección de memoria especificada. En el contexto de MMIO, si la dirección corresponde a un dispositivo mapeado, sw modificará el registro del dispositivo (por ejemplo, activará un sensor o actualizará un control).

2) ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

Diferencia principal entre Memory-Mapped I/O y Entrada/Salida por Puertos:

Memory-Mapped I/O (MMIO): Los dispositivos de E/S se mapean directamente en el espacio de direcciones de memoria. El procesador accede a los dispositivos usando las mismas instrucciones de memoria como lw (load word) y sw (store word).

Entrada/Salida por Puertos: Los dispositivos de E/S están conectados a puertos específicos, y la comunicación con estos dispositivos se realiza mediante instrucciones especiales como IN y OUT, que son diferentes de las instrucciones de memoria.

Ventajas y desventajas:

Memory-Mapped I/O:

Ventajas: - Simplificación en la programación, ya que se usan las mismas instrucciones de memoria. - Acceso rápido y flexible a los dispositivos, similar a la memoria. - Posibilidad de compartir espacio de direcciones entre memoria y dispositivos.

Desventajas: - Puede hacer que el espacio de direcciones se agote más rápido. - Posibles conflictos si las direcciones no están bien gestionadas.

Entrada/Salida por Puertos:

Ventajas: - Separación clara entre dispositivos de E/S y memoria. - Puede manejar múltiples dispositivos de forma independiente.

Desventajas: - Requiere instrucciones especiales, lo que hace la programación más compleja. - El acceso a los dispositivos puede ser más lento debido a la necesidad de instrucciones separadas.

¿Por qué MIPS32 utiliza principalmente Memory-Mapped I/O?

MIPS32 utiliza Memory-Mapped I/O principalmente por su simplicidad y eficiencia. Al tratar a los dispositivos de E/S como si fueran direcciones de memoria, el procesador puede acceder a ellos con las mismas instrucciones que usa para la memoria, lo que reduce la complejidad del hardware y la programación. Además, MMIO permite aprovechar completamente el espacio de direcciones y es compatible con arquitecturas modernas.

3) En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Cuando dos dispositivos mapeados en memoria utilizan direcciones solapadas, pueden surgir varios problemas, como:

- **Confusión en el acceso:** El procesador no sabría a qué dispositivo está accediendo, ya que ambas direcciones corresponden a diferentes dispositivos, pero están en el mismo espacio de direcciones. - **Sobreescritura de datos:** Si ambos dispositivos están mapeados a la misma dirección, al intentar leer o escribir, los datos de uno pueden sobrescribir los del otro, causando errores y mal funcionamiento. - **Inestabilidad del sistema:** Si los dispositivos no están bien diferenciados en la memoria, pueden ocurrir comportamientos inesperados, ya que el procesador podría leer o escribir datos incorrectos.

Cómo se evita este conflicto:

- **Asignación de direcciones únicas:** El sistema debe asegurarse de que cada dispositivo tenga direcciones únicas dentro del espacio de direcciones de memoria. Esto se logra mediante un proceso de planificación de direcciones. - **Segmentación del espacio de direcciones:** En muchos sistemas, se asignan rangos específicos de direcciones para MMIO, asegurando que no se solapen. Por ejemplo, se pueden reservar las últimas direcciones del espacio de memoria para MMIO y dividir ese rango entre los diferentes dispositivos. - **Uso de un mapa de direcciones:** Se debe usar un mapa de direcciones para gestionar el espacio de direcciones MMIO. Este mapa asegura que no haya conflictos entre las direcciones de los dispositivos de E/S y las direcciones de memoria del programa.

4) ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Memory-mapped I/O (MMIO) simplifica el diseño del conjunto de instrucciones de un procesador por varias razones:

- **Uso de las mismas instrucciones:** En un sistema MMIO, los dispositivos de E/S se tratan como ubicaciones de memoria. Esto significa que no se necesitan instrucciones adicionales específicas para manejar dispositivos de E/S. El procesador solo usa las instrucciones estándar de carga y almacenamiento, como lw (load word) y sw (store word), para interactuar tanto con la memoria como con los dispositivos de E/S. - **Simplificación del hardware:** No es necesario implementar instrucciones especiales o unidades de hardware dedicadas para gestionar la entrada/salida. Al no tener que diferenciar entre instrucciones de memoria y de

E/S, el conjunto de instrucciones se reduce, lo que facilita el diseño y mejora la eficiencia. - **Acceso uniforme:** Los dispositivos de E/S y la memoria comparten el mismo espacio de direcciones, lo que simplifica la organización y el manejo del sistema, permitiendo al procesador acceder a ambos con las mismas operaciones.

Instrucciones adicionales necesarias con E/S por puertos:

Si se usara E/S por puertos en lugar de MMIO, se necesitarían instrucciones adicionales para gestionar la interacción con los dispositivos de E/S. Estas instrucciones podrían incluir:

- **Instrucciones IN y OUT:** - IN: Leer un dato desde un puerto de entrada hacia un registro. - OUT: Escribir un dato desde un registro hacia un puerto de salida.

Estas instrucciones son específicas para acceder a los dispositivos de E/S y requieren que el procesador tenga un conjunto de instrucciones diferente o especializado para la E/S.

- **Instrucciones de control de puertos:** Puede ser necesario tener instrucciones para configurar los puertos, habilitar o deshabilitar la comunicación, o gestionar interrupciones relacionadas con los dispositivos de E/S.

- **Instrucciones de direccionamiento de puertos:** En lugar de acceder a la memoria, el procesador tendría que utilizar instrucciones específicas para leer o escribir en direcciones de puertos físicas, lo que implica un mayor número de instrucciones y una complejidad adicional.

5) ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

¿Qué ocurre a nivel del bus cuando se accede a una dirección de un dispositivo?

Cuando el procesador accede a una dirección de un dispositivo mapeado en memoria (MMIO):

- **Dirección en el bus:** El procesador coloca la dirección en el bus de direcciones. - **Control de acceso:** El bus de control indica si es una lectura o escritura. - **Acceso al dispositivo:** Si la dirección corresponde a un dispositivo, el controlador MMIO redirige la solicitud al dispositivo de E/S, no a la RAM. - **Transacción de datos:** Si es lectura, los datos del dispositivo se colocan en el bus de datos; si es escritura, el valor se envía al dispositivo.

¿Cómo sabe el hardware que debe acceder a un periférico?

El hardware sabe a qué acceder mediante:

- **Mapa de direcciones:** Los dispositivos de E/S están en un rango específico de direcciones, distinto de la RAM. - **Decodificador de direcciones:** Compara la dirección solicitada con los rangos predefinidos para saber si es un dispositivo o memoria. - **Controlador MMIO:** Se encarga de redirigir la solicitud al dispositivo correspondiente.

6) ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En sistemas modernos, la respuesta es no de forma directa. Si un programa de usuario intentara acceder a una dirección física de un periférico sin permiso, el hardware dispararía una excepción de seguridad.

Para evitar esto, se utilizan mecanismos de abstracción:

- **La MMU (Unidad de Gestión de Memoria):** Es un componente de hardware que actúa como un "traductor". Su trabajo es convertir las direcciones virtuales (las que ve el programa) en direcciones físicas (las que realmente existen en el hardware). - **Mapeo de Espacio Virtual:** El sistema operativo no le da al programa acceso a la memoria real, sino que le crea un "mapa"propio (espacio virtual). Si el programa intenta acceder a una dirección que no está en su mapa asignado —como la dirección de control del tensiómetro—, la MMU detecta que esa traducción no es válida para ese usuario y bloquea la instrucción. - **Modos de Privilegio:** El procesador MIPS utiliza niveles de ejecución. El acceso a las direcciones de hardware suele estar restringido al Modo Kernel. Un programa de usuario solo puede interactuar con el dispositivo a través de "llamadas al sistema"(syscalls), dejando que el sistema operativo haga el trabajo sucio de forma segura.

7) ¿Qué técnicas se emplean para evitar esperas activas innecesarias al interactuar con dispositivos?

La .espera activa"(como el bucle que usamos en el código del controlador) consume ciclos de CPU que podrían aprovecharse en otras tareas. Para optimizar esto, existen dos alternativas principales:

- **Interrupciones:** Es la técnica más eficiente. El procesador le dice al dispositivo: .avísame cuando termines". Mientras tanto, la CPU ejecuta otros procesos. Cuando el dispositivo tiene los datos listos, envía una señal eléctrica (IRQ) que pausa la ejecución actual para atender al periférico. - **Acceso Directo a Memoria (DMA):** Se utiliza cuando hay que mover grandes volúmenes de datos. Un controlador de DMA se encarga de transferir la información directamente entre el dispositivo y la RAM sin que la CPU intervenga en cada palabra transferida, liberándola por completo durante el proceso.

8) Análisis y discusión de los resultados

Para analizar y discutir los resultados, primero debemos evaluar el mejor caso, un caso de error, y el peor caso de cada procedimiento:

- Sensor de Luminosidad

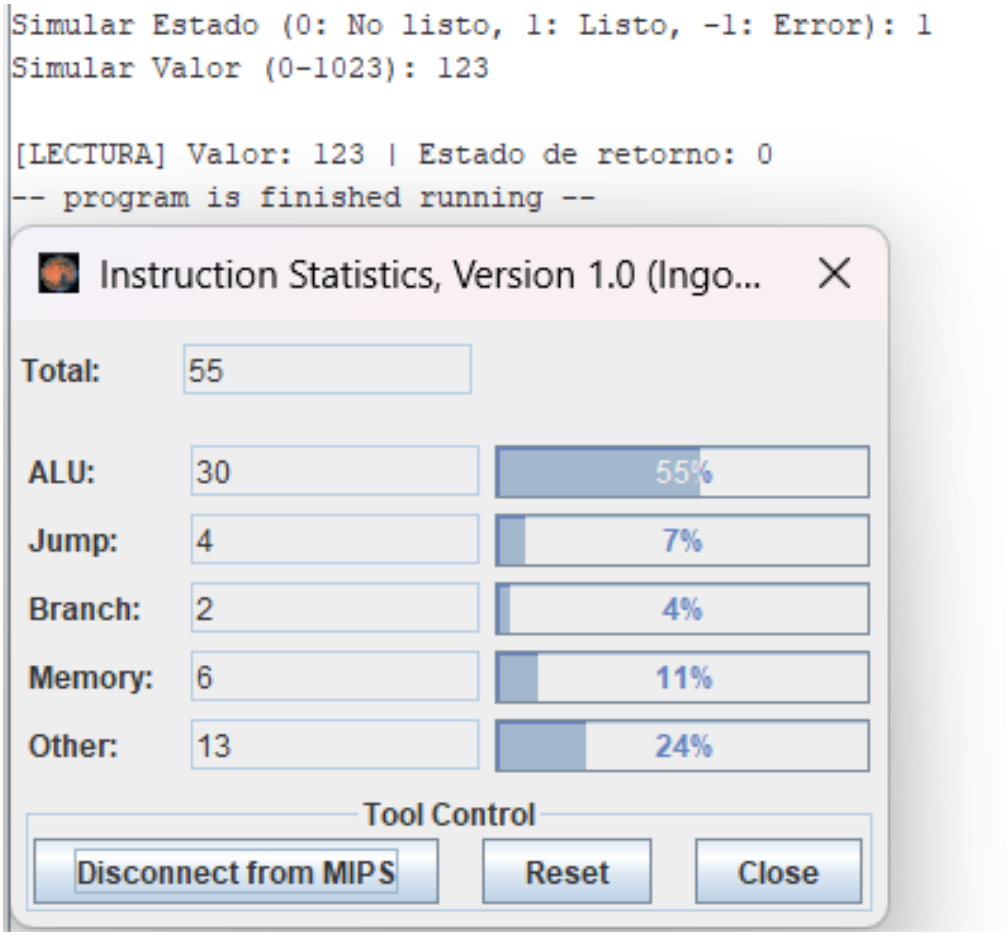
El mejor caso ocurre cuando el registro LuzEstado contiene el valor 1 al momento de la primera consulta del procesador.

Métrica de Instrucciones: Se registró un total de 55 instrucciones ejecutadas desde el inicio del programa hasta su finalización exitosa.

Flujo de Control: El procedimiento InicializarSensorLuz entra en la etiqueta EsperarSensor, realiza una única lectura de memoria (lw) y, al comparar el valor con cero (beq), la condición resulta falsa. Esto permite que el programa rompa.[el] bucle de espera en su primera iteración.

Acceso a Datos: El procedimiento LeerLuminosidad procede a cargar el valor de la dirección LuzDatos y retorna el código de éxito (0) en el registro $v1.

Este escenario representa la máxima eficiencia operativa del sistema. Al no requerir ciclos adicionales de espera (polling), el procesador minimiza el consumo de recursos y el tiempo de respuesta (latencia). La diferencia de solo 2 instrucciones respecto al caso de error demuestra que el costo computacional de procesar un dato válido es mínimo, limitándose únicamente al tiempo de acceso a la memoria RAM para traer el valor de luminosidad.



El caso de error se presenta cuando el registro LuzEstado devuelve el valor -1. Este valor es una señal de diagnóstico que indica que el hardware ha detectado un fallo interno crítico (como un cortocircuito o una descalibración) y no puede proporcionar una lectura válida.

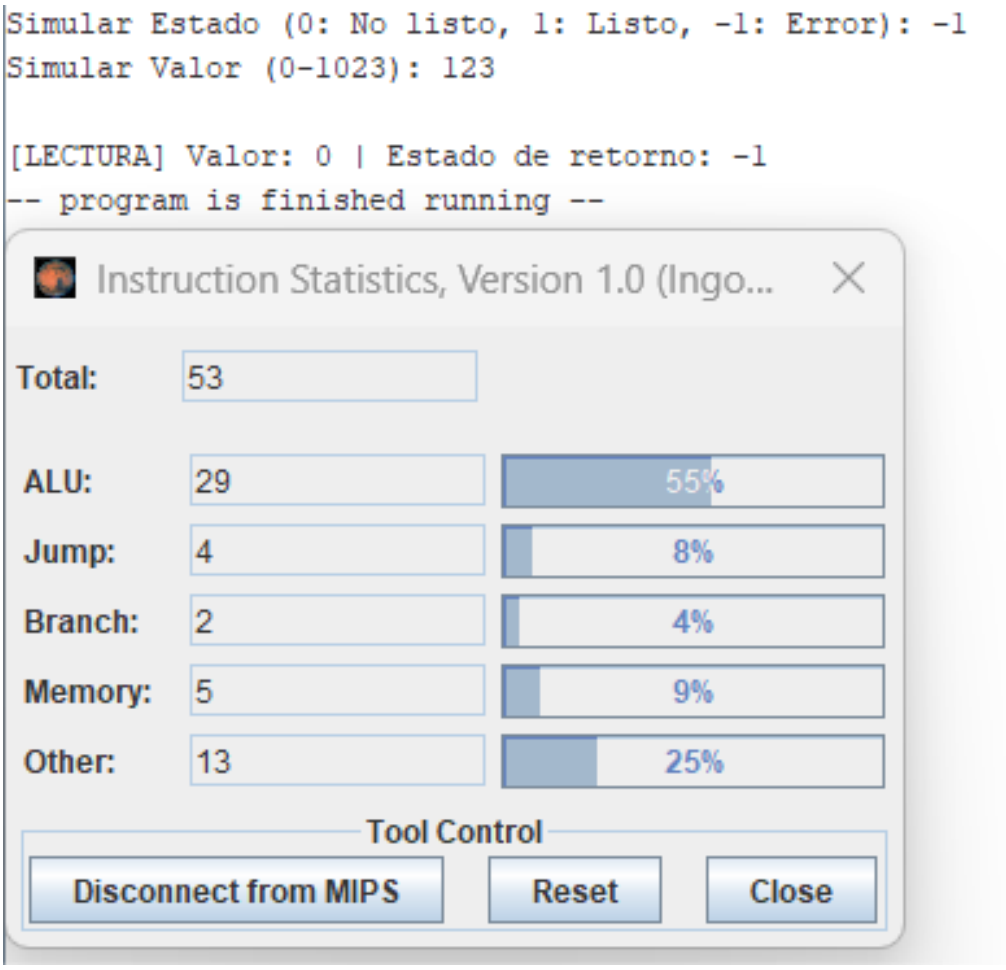
Métrica de Instrucciones: Se registró un total de 53 instrucciones, el número más bajo de todos los escenarios probados.

Flujo de Control: El procedimiento InicializarSensorLuz detecta el -1 y sale del bucle de espera inmediatamente, ya que el valor es distinto de cero.

Al entrar en LeerLuminosidad, la instrucción beq $t2, $t4, Error resulta verdadera. El procesador realiza un salto (branch) hacia la etiqueta de error, omitiendo por completo las instrucciones de carga de memoria de la luminosidad real.

Manejo de Datos: El programa carga por software un valor de "seguridad"(0) en el registro de datos $v0 y reporta el código de falla (-1) en el registro de estado $v1.

Es notable que el caso de error sea más rápido (en 2 instrucciones) que el mejor caso. Esto ocurre debido a una optimización lógica: si el sensor reporta un error, no tiene sentido que el procesador gaste ciclos de reloj intentando acceder a la dirección LuzDatos, ya que esos datos no son confiables. El sistema prioriza la integridad de los datos y la velocidad de respuesta del diagnóstico sobre la obtención del valor de luminosidad.



El peor caso absoluto ocurre cuando el registro LuzEstado contiene el valor 0. En la lógica

del sensor, esto indica que el dispositivo está .ºcupado.º "No Listo". A diferencia de los casos anteriores, este escenario no tiene un tiempo de finalización definido por el software.

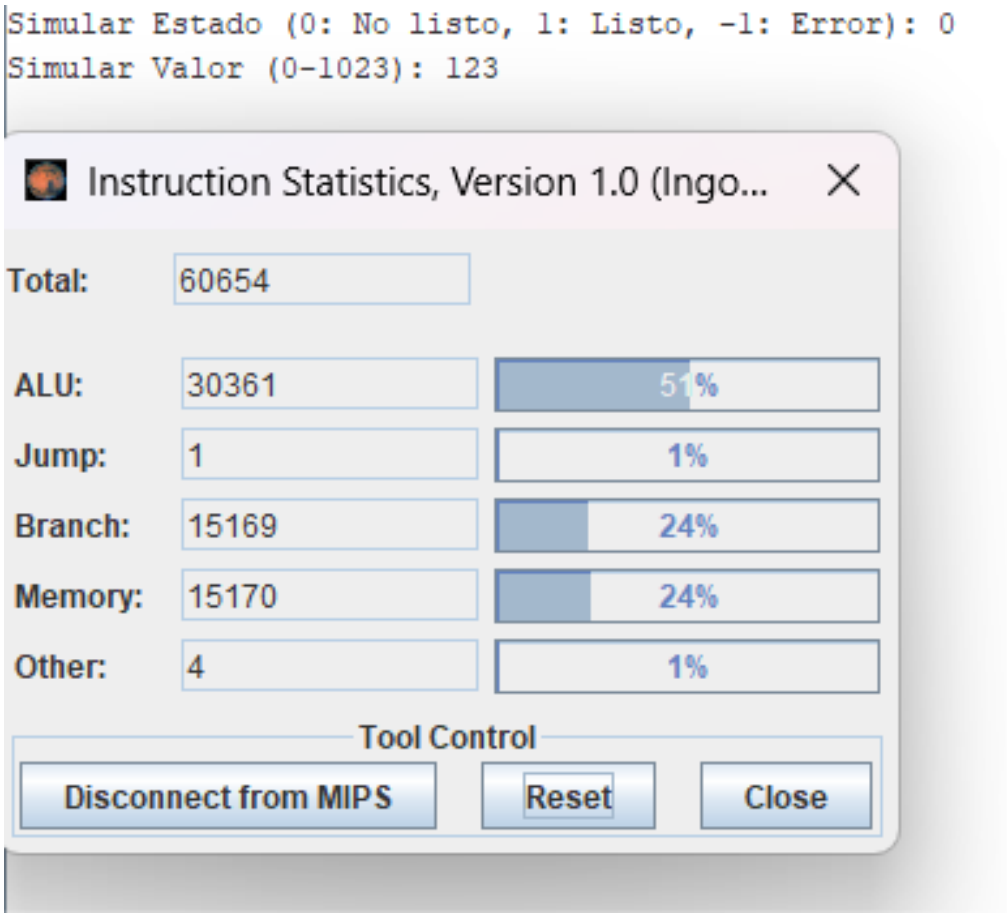
Métrica de Instrucciones: El número de instrucciones ejecutadas es indeterminado e infinito. Mientras el sensor no cambie su estado, el contador de instrucciones en MARS aumentará sin detenerse.

Flujo de Control (Polling): El procesador entra en un bucle de espera activa. En cada iteración, ejecuta la siguiente secuencia:

- Carga la dirección de memoria (la). - Lee el valor del registro de estado (lw). - Evalúa la condición de salto (beq). Al ser el valor igual a cero, el salto es verdadero, devolviendo el contador de programa (PC) al inicio del bucle.

Consumo de Recursos: El procesador opera al 100 % de su capacidad, realizando ciclos de lectura constantes sobre el bus de datos sin producir ningún progreso en la lógica del programa principal.

Este resultado revela la vulnerabilidad inherente de los sistemas que dependen de la sincronización por polling. Si bien el código cumple con el requerimiento de .esperar a que el sensor esté listo", un fallo en el hardware que mantenga el estado en 0 de forma permanente provocaría un Livelock (bloqueo activo). El sistema queda .atrapadoz pierde su capacidad de respuesta.



- Sensor de Presión

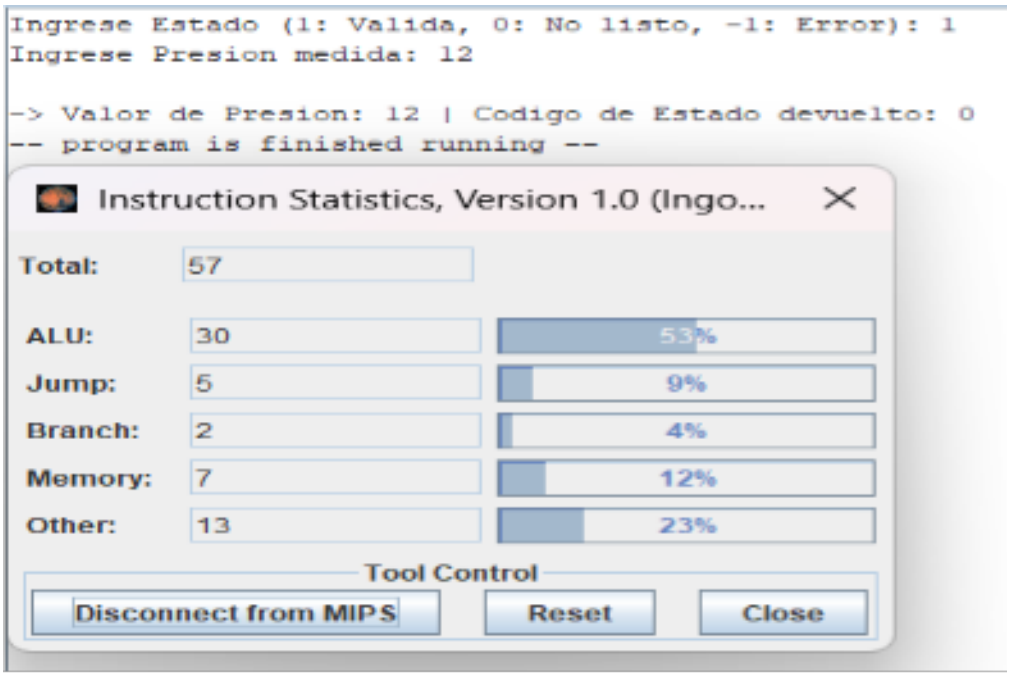
El mejor caso ocurre cuando el sensor tiene una lectura válida (Estado = 1) desde la primera consulta.

Flujo lineal: El programa inicializa el sensor, verifica el estado y, al ver que es correcto, salta directamente a extraer el dato.

Métrica de Instrucciones: Es el camino más corto para este sensor.

Eficiencia: No se ejecuta la lógica de reintento ni se vuelve a llamar a la función de inicialización, ahorrando ciclos de reloj innecesarios.

Este caso representa la operatividad óptima. El código garantiza la integridad del sistema al preparar la pila, pero no penaliza el rendimiento porque detecta el éxito de inmediato y evita el bloque de reintento".



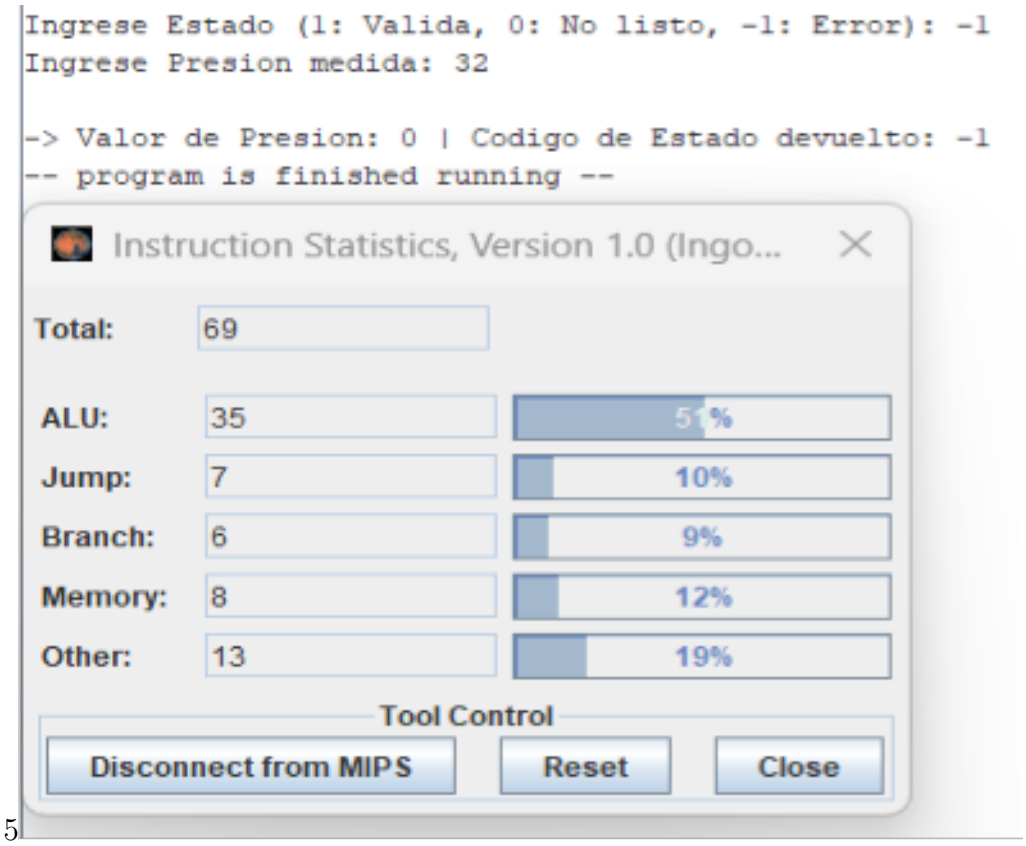
El caso de error ocurre cuando el primer chequeo del estado devuelve -1. El sistema intenta recuperarse por sí mismo sin pedirle nada al usuario.

Ejecución Invisible: El procesador detecta el error y vuelve a llamar a la función de inicialización de forma automática.

Persistencia del Error: Como el usuario no ingresa un nuevo valor durante el milisegundo que dura el reintento, el programa vuelve a leer el mismo -1 que ya estaba en la memoria.

Costo de Instrucciones: Es el camino que más instrucciones gasta (sin contar el bucle infinito) porque recorre el código de inicialización dos veces.

El reintento es una operación de software transparente. Aunque el resultado final acabe siendo error (porque el valor en memoria no cambió), el programa demuestra su capacidad de intentar restablecer el hardware antes de rendirse y devolver el código de fallo definitivo (-1 en $v1).



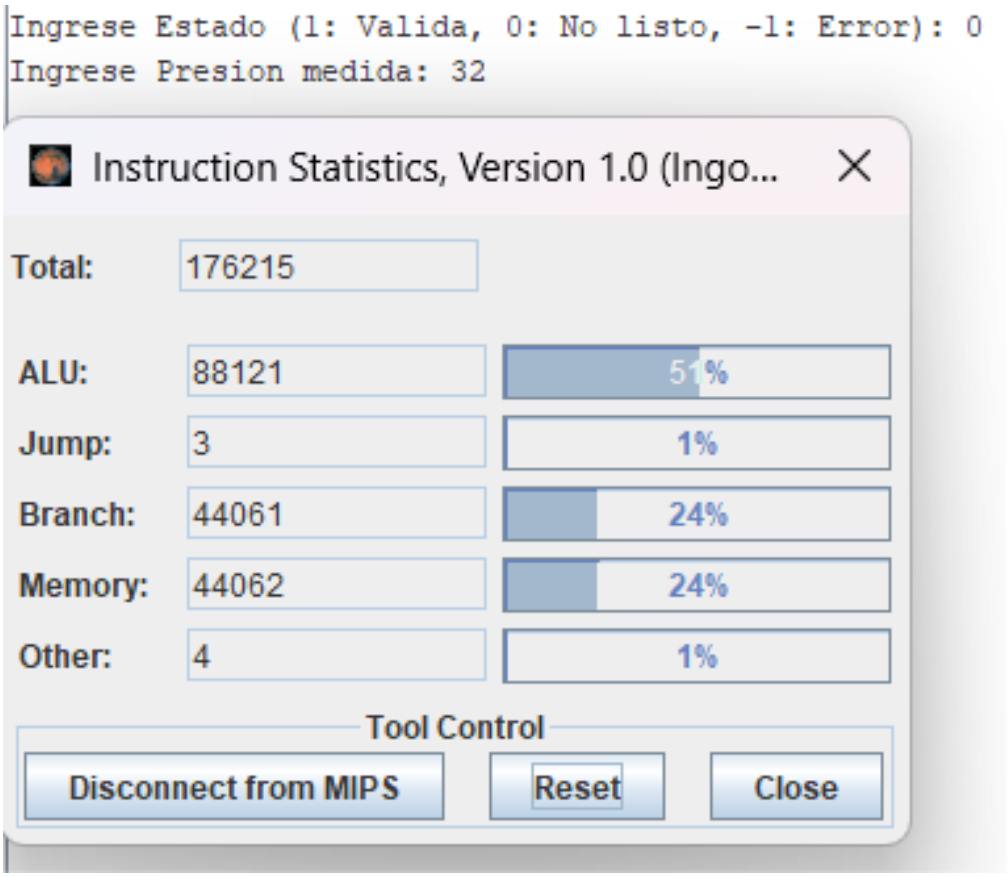
El peor caso ocurre cuando se ingresa el valor 0 (No listo) en el registro de estado.

Bucle Infinito (Polling): El procesador entra en la etiqueta EsperarPresion y se queda ejecutando las mismas instrucciones (la, lw, beq) una y otra vez.

Bloqueo del Sistema: Como bien señalas, el programa no vuelve a pedirle nada al usuario ni avanza; se queda .ªtrapado.ᵉˢperando que el valor en la dirección de memoria PresionEstado cambie a algo distinto de cero.

Instrucciones: El contador de instrucciones tiende a infinito.

Es el peor escenario posible porque causa un Livelock. El procesador consume el 100 % de sus ciclos sin realizar ninguna tarea útil, y el programa pierde totalmente la capacidad de respuesta, quedando a la merced de que un evento externo (hardware real) modifique la memoria para poder salir del bucle.



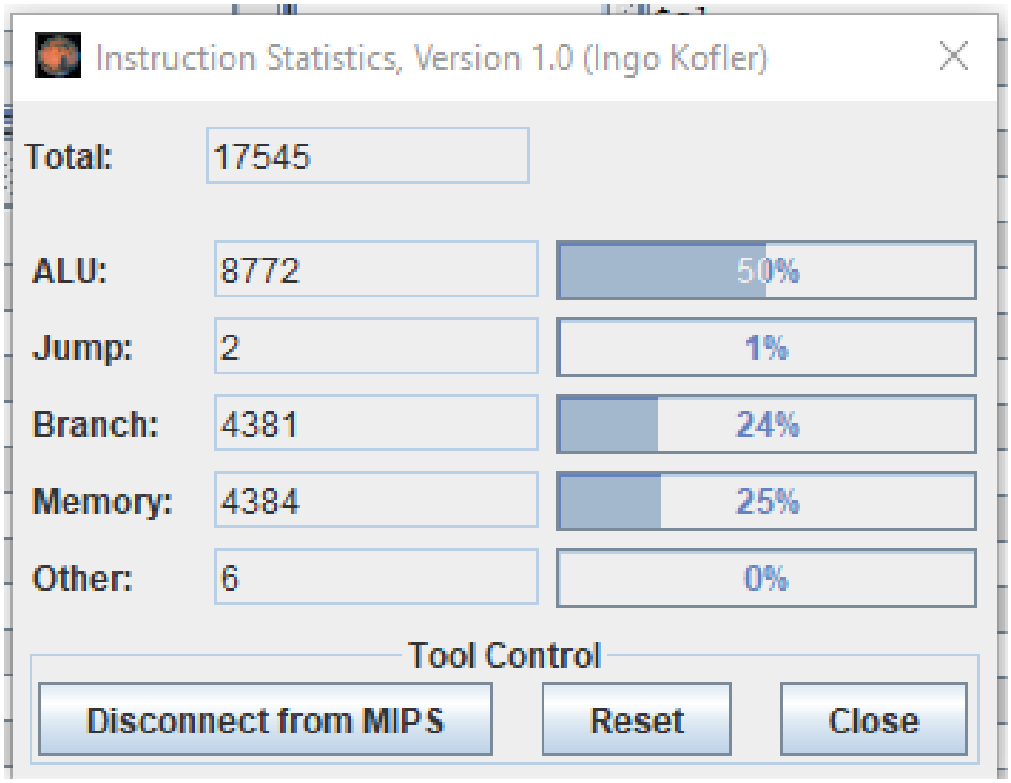
- Sensor de Tensión Arterial

El mejor caso ocurre cuando el sensor tiene datos disponibles desde la primera consulta del registro TensionEstado.

Flujo lineal: El programa inicializa el sensor escribiendo en TensionControl, verifica el estado y, al detectar que los datos ya están listos, salta directamente a leer TensionDato para la presión sistólica y diastólica, evitando ciclos adicionales de espera.

Métrica de instrucciones: Cada bucle de espera se ejecuta una sola vez, resultando en un número mínimo de instrucciones ejecutadas. Aunque existe un tiempo mínimo de latencia debido a la secuencia de instrucciones y acceso a memoria, este caso representa el camino más corto práctico para el procedimiento.

Eficiencia: No se repiten ciclos de polling innecesarios, por lo que la CPU realiza únicamente el trabajo útil de inicialización y lectura de datos. Representa el rendimiento óptimo del algoritmo, con mínima ocupación de ciclos de reloj.

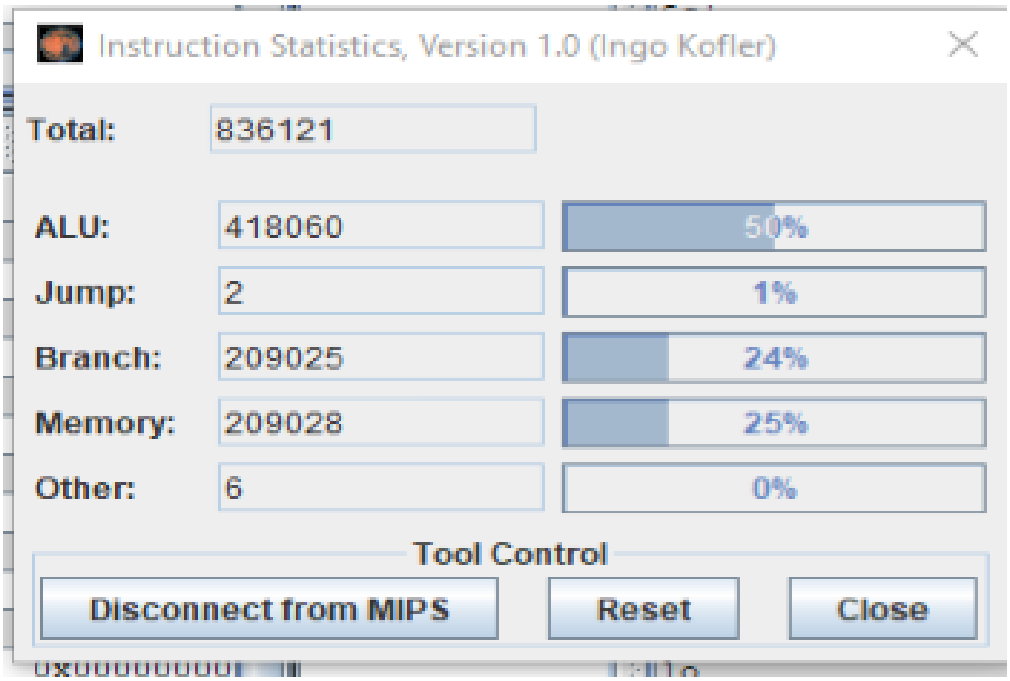


El peor caso ocurre cuando el sensor tarda en generar los datos o, en la simulación, cuando el usuario demora en presionar las teclas correspondientes.

Flujo repetitivo: Tras iniciar la medición, el programa entra en los bucles esperar sistolica y esperar diastolica, ejecutando repetidamente las instrucciones de lectura de estado, máscara y salto condicional hasta que los datos estén disponibles.

Métrica de instrucciones: Cada iteración adicional del bucle aumenta el número total de instrucciones ejecutadas de forma proporcional al tiempo de espera. Por lo tanto, el número de instrucciones puede crecer significativamente comparado con el mejor caso.

Eficiencia: La CPU permanece ocupada en polling durante todo el tiempo de espera, sin realizar trabajo útil adicional. Esto evidencia que la eficiencia depende directamente del tiempo de respuesta del dispositivo, y muestra la limitación del enfoque de espera activa.



Un posible caso de error del algoritmo ocurre cuando el sensor nunca cambia el valor del registro TensionEstado a 1. En esta situación, el programa permanece indefinidamente dentro del bucle de espera (polling), generando un ciclo infinito.

Dado que el procedimiento depende completamente de la disponibilidad del dato, la ausencia de actualización del estado impide la lectura de la presión sistólica y diastólica. Esto provoca que el programa no retorne resultados y que el procesador continúe ejecutando instrucciones de verificación sin progreso funcional.

Este comportamiento evidencia una limitación del mecanismo de espera activa, ya que no contempla un mecanismo de tiempo límite o manejo de errores. En sistemas reales, este problema podría mitigarse mediante el uso de interrupciones o temporizadores que permitan detectar fallos en la comunicación con el dispositivo.